Reinforced Genetic Algorithm Learning For Optimizing Computation Graphs

딥러닝논문읽기 5기

강화학습팀 : 이도현, 백승언, 김현성, 주정헌(발표자)

발표일 : 2023년 3월 26일

Introduction

- **Growing interest in the use of optimizing static compilers for neural network computation graphs**
  - Deep learning frameworks represent neural network models as **computation graphs**.
- **In model parallelism, a computation graph can be executed using multiple devices in parallel.**
  - Graph
    - Nodes of the graph are computational tasks
    - Directed edges denote dependencies between them
  - Tasks
    - Jointly optimizing over Placement: which nodes are executed on which devices
    - Schedule: the node execution order
  - Objectives
    - minimize running time, peak memory usage

Background

- → Performance Model
  - → Computation graph setting
  - → Finding an assignment of **ops** to **devices** and an overall **schedule**
    - → to find a feasible way to run a large computation graph
    - → minimizing the runtime subject to a constraint on the peak memory used on any device.

Background

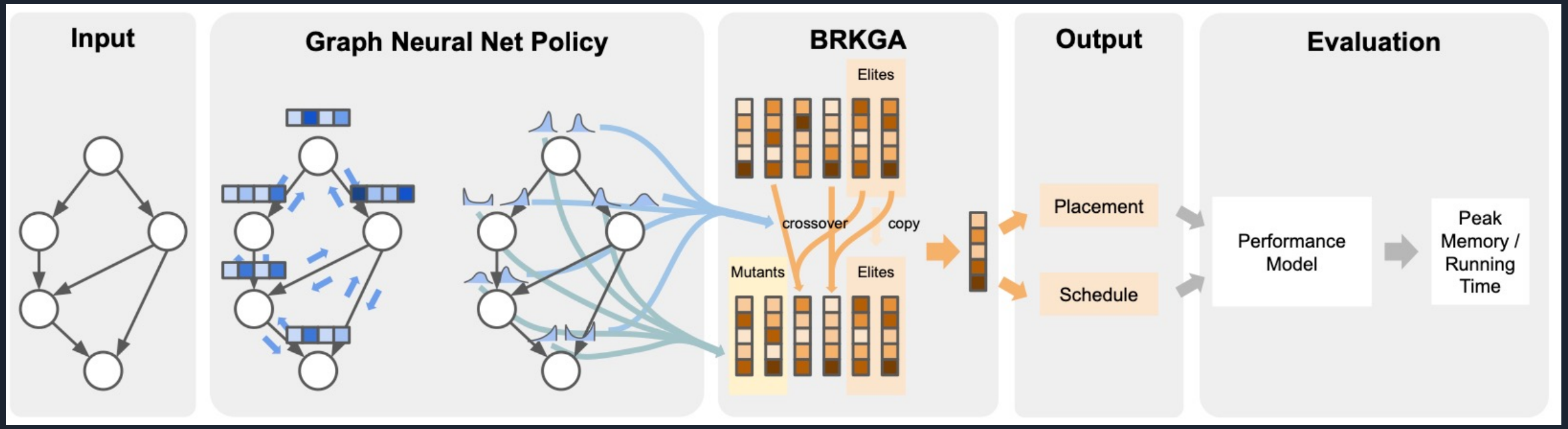
- → BRKGA(BIASED RANDOM-KEY GENETIC ALGORITHM) -> aka. “Biased”
  - → Initial Population : Sample some chromosomes
  - → Evolution step
    - → Ranking the chromosomes by a function
    - → Constructing the next generation from these
      - → Copying elite chromosomes from last generation
      - → Select two parents (one is non-elite and one is from elites)
      - → Applying crossover to generate a new chromosome given two parents

Background

- → Contextual Bandit Formulation : RL problem with an agent interacts with environments and rewards.
  - → Goal: Maximize the cumulative rewards it receive over time
  - → Explanation : Receiving additional information from contextualized features. Adaptation to the changing environments

Q & A

REGAL(Key idea)



Preprocessing Algorithms as GNN

1. Computation graph as input and output node-specific distributions
2. Objective values as a reward signal in a contextual bandit approach

Optimization Algorithms : BRKGA

1. Run to completion with input-dependent distribution

Experiments Results

- → Setting
  - → Device : 2 homogeneous devices with 16GiB of memory each synchronous tensor transfers with zero cost
  - → Model : Train a separate neural network for each task-dataset pair for the case of two devices
  - → Dataset : 372 topologically-unique real-world TensorFlow graphs by mining ML jobs on Google’s internal computer cluster.

Experiments Results

→ Results

Table 1: Comparison of methods on the TensorFlow and Synthetic test sets. Results are averages over test set graphs. Higher is better for % Improvement over BRKGA5K, and lower is better for % Gap from best known. Note: CP-SAT, an enumerative algorithm, is run for up to 24 hours only to establish provably global optima (if possible) for evaluation purposes.

Algorithm	TF Runtime test set		TF Peak Memory test set		Synthetic Runtime test set	
	% Improv. over BRKGA 5K	% Gap from best known	% Improv. over BRKGA 5K	% Gap from best known	% Improv. over BRKGA 5K	% Gap from best known
CP-SAT 24hr	15.85%	1.00%	-1.48%	8.06%	19.50%	0.00%
GP + DFS	-37.32%	66.98%	-6.51%	14.77%	-55.8%	93.66%
Local Search	-1.66%	22.63%	0.63%	7.24%	0.08%	24.60%
BRKGA 5k	0.00%	20.19%	0.00%	7.98%	0.00%	24.63%
Tuned BRKGA	3.20%	16.40%	0.80%	7.11%	3.11%	20.76%
GAS	3.79%	15.24%	0.16%	7.67%	0.80%	23.48%
IDRS	-6.87%	28.60%	-3.16%	12.39%	-12.12%	39.72%
REGAL	**7.09%**	**11.04%**	**3.56%**	**4.44%**	**4.81%**	**18.57%**

Experiments Results

→ Results (REGAL vs BRKGA)

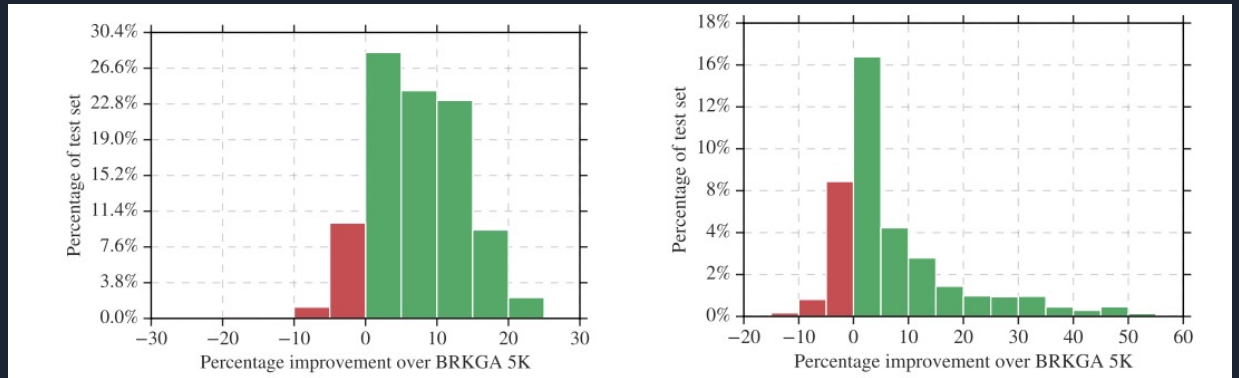


Fig. 2: Histogram of percent improvements in objective value on the TensorFlow runtime (left) and peak memory (right) datasets for test graphs on which REGAL is better (green) and worse (red) than BRKGA. (Ties are omitted from the figure for clarity but are included in the histogram percentage calculation.)

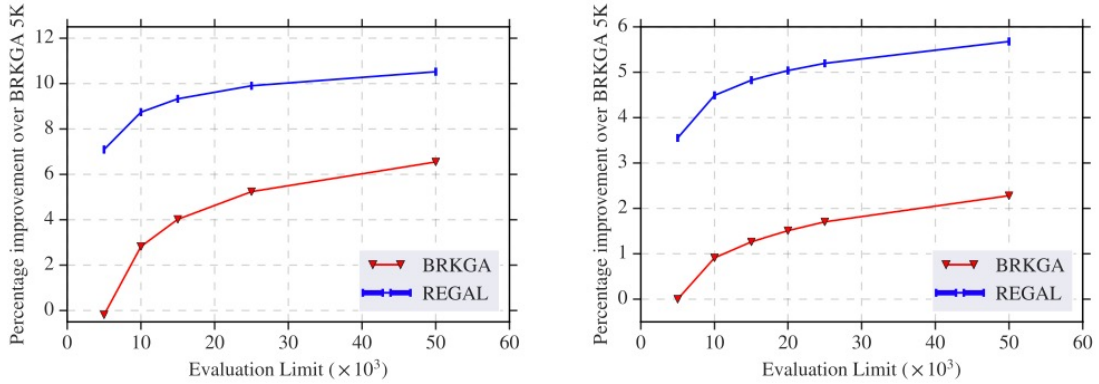


Fig. 3: Average percent improvement over BRKGA 5K given by REGAL and BRKGA on the TensorFlow test set for running time (left) and peak memory (right) as the evaluation limit is increased.

Contribution & Conclusion

1. First Demonstration

This work is the first to demonstrate learning a policy for jointly optimizing placement and scheduling that generalizes to a broad set of real-world TensorFlow graphs.

2. Use of Graph Neural Network

This work uses a graph neural network to predict mutant sampling distributions of a genetic algorithm, specifically BRKGA, for the input graph to be optimized.

3. Extensive Experiment

This work compares extensively to classical optimization algorithms, such as enumerative search, local search, genetic search, and other heuristics, and analyze room-for-improvement in the objective value available to be captured via learning.

Q & A